EBU6304 Software Engineering Group Project

2025-26

Group number	28	
member 1	QM no: 231220965	Name: Fu Bo
member 2	QM no: 231223014	Name: Zhang Yutong
member 3	QM no: 231221766	Name: Hu Weize
member 4	QM no: 231222936	Name: Yan Junyi
member 5	QM no: 231222626	Name: Zeng Yihua
member 6	QM no: 231220943	Name: Tian Liyuan

First report

1.Requirements

1. Fact-Finding Techniques
Fact-finding was conducted to identify and validate the functional, non-functional, and role-specific requirements for the TA recruitment system. The following structured techniques were used and tailored to the project's stakeholder roles and business needs:
1.1 Stakeholder Interviews

- **Target Participants**: TA applicants (end users for profile creation and job applications), MOs (core users for job posting and application review), and school Admins (system administrators for oversight and data management).
- **Focus**: Uncovered role-specific pain points (e.g., TA difficulty finding skill-matched jobs, MO inefficiency in screening applicants) and core functional needs (e.g., secure login, profile management, application tracking).
- **Outcome**: Defined role-based permission boundaries and core user stories (e.g., *US-CORE-001 Create TA Profile*, *US-CORE-005 View Applicants*).

1.2 Use Case Analysis

- **Approach**: Mapped end-to-end business processes for the TA recruitment lifecycle (profile creation/CV upload → job posting → application → review/delete → workload monitoring) and decomposed each process into actionable use cases.
- **Outcome**: Translated processes into structured user stories with clear **actors** (TA/MO/Admin), **triggers**, and **expected outcomes**, forming the foundation of the Product Backlog. Completed the project's UML diagram, visualized Product Backlog. The use cases for 'Viewing Job Details' and 'Tracking Application Status' were specifically designed to address the instructor's mandate for automatic notifications and deadline highlights, ensuring applicants can make rational decisions based on working hours.

1.3 Requirements Validation with Prototype Walkthroughs

- **Approach**: Created low-fidelity prototypes for core features (e.g., login interface for the first version) and walked stakeholders through key user flows.
- **Outcome**: Refined acceptance criteria for user stories (e.g., adding *5-minute account lock after 3 failed logins* for the login feature) and eliminated ambiguous requirements.

1.4 Document Analysis & Benchmarking

- **Approach**: Reviewed existing school TA recruitment documentation (e.g., job posting

templates, application forms) and benchmarked similar recruitment systems for standard features (e.g., password change, data export) and best practices (e.g., skill-based job recommendation). Benchmarking was supplemented with expert advice from TAs regarding multimodal models, leading to the inclusion of AI-assisted resume review features to enhance efficiency.

- **Outcome**: Ensured alignment with institutional processes and added practical supplementary features (e.g., *US-LOW-003 Export Recruitment Data*, *US-MID-003 AI Job Recommendation*).

1.5 Non-Functional Requirements Elicitation

- **Focus**: Gathered non-functional requirements critical to the system (e.g., **file format constraints** [only TXT/CSV, no database], **security rules** [session management, account lock], **performance** ["Delete" button is enabled only for "Pending" applications and before the job deadline.]).
- **Outcome**: Aligned with instructor requirements, defined key non-functional requirements (NFRs), including automatic notifications, deadline highlighting, and enforced working-hour rules, to satisfy end-user needs and administrative standards. All critical non- functional logic was embedded into the acceptance criteria of core user stories (e.g., *Unlogged users redirected to login page* for *US-SYS-001 System Login*).

2.Team Collaboration & Decision Making

2.1 Team Member Roles and Collaboration Process

Team members were actively involved in all research activities. Bo Fu and Yutong Zhang conducted interviews with TAs and MOs respectively, while other team members researched similar online systems (see Supporting Material). After gathering all stories, we held collaborative discussions to evaluate and prioritize features.

The division of labor was as follows:

- **Yutong Zhang and Bo Fu:** Responsible for stakeholder interviews and initial data collection
- **Weize Hu and Yihua Zeng:** Developed prototypes based on project UML diagrams created by Yutong Zhang
- **Yutong Zhang, Bo Fu, and Liyuan Tian:** Collaborated on report writing
- **Junyi Yan:** Focused on building the system's foundational framework. This collaborative approach ensured that all team members contributed to the research and decision-making process.

2.2 Conflict Resolution: Standardized Resume vs. PDF Upload

The Conflict: During internal group discussions, a conflict arose between developers and end-users (TAs). The development team initially favored a "Standardized In-System Resume" to ensure data uniformity and reusability. However, TAs (from the Supporting Material) strongly advocated for PDF support, citing convenience and mainstream usage.

The Decision (Trade-off): After weighing the pros and cons, the team prioritized User Experience (UX) over technical convenience. We decided to prioritize PDF upload functionality as the core requirement and relegated the standardized resume system to a future enhancement plan.

2.3 Feature Prioritization based on Stakeholder Value

The Process: The team collaborated to evaluate the "AI Job Recommendation" feature. While the instructor emphasized its value, the team noted the high effort required (US-MID-003).

The Decision: Using the Value-Effort Matrix, we agreed to classify this as a "Could-Have" (Medium Priority) feature, ensuring it would not delay the delivery of the core MVP (Minimum Viable Product).

3. Iteration Plan

The project adopts an agile Sprint-based iteration plan with a fixed timeline and role-based feature delivery. Iterations are prioritised to deliver minimum viable product functionality first, followed by enhanced and administrative features, ensuring early demo and user testing for core workflows.

Phase 0: Initial setup

Key Goals:

- Group and GitHub Repository Setup
- Story writing workshop Prototype with user feedback

Deliverables:

- Product Backlog (core user stories)
- Low-fidelity prototype
- Working Software version 1 (functional core workflow)
- GitHub repository with initial codebase

Phase 1: Sprint 1 – Core MVP Functionality (Highest Priority)

Key Goals:

- Deliver the end-to-end minimum viable workflow for TA recruitment:
    1. User authentication (login, logout)
    2. TA profile creation & CV/resume upload
    3. MO job posting
    4. TA browse open jobs & submit applications
    5. MO view applicants for their own jobs
- Non-functional constraints: File I/O (TXT/CSV only), basic security rules (login attempt lock, session management)

Deliverables:

- Working Software version 1
- Demonstration and Viva
- Updated GitHub repository

Phase 2: Sprint 2 – Key Enhanced Functionality (High Priority)

Key Goals:

- Complete the recruitment review cycle and add management features:
    1. MO application review & status updates (Accept/Reject)
    2. TA application status checking
    3. MO job editing & closing
    4. MO recruitment information publication
    5. Admin view all jobs & TA workload monitoring

Deliverables:

- Working Software version 2 (full recruitment workflow)
- Demonstration & viva for intermediate assessment
- Updated GitHub repository

Phase 3: Sprint 3 – Supplementary Convenience Functions (Medium Priority)

Key Goals:

- Add extended user experience & administrative functions:
    1. TA registration & login (phone/email)
    2. TA favorite/save interested positions
    3. MO post job announcements for hired TAs
    4. Admin manage user accounts & permissions

5. MO filter applicants by skill match rate
6. AI-powered TA workload balance suggestions (Admin)

Deliverables:

- Working Software version 3 (system with enhanced UX)
- Updated GitHub repository

Phase 4: Sprint 4 – Final Completion & Polishing (Low Priority)

Key Goals:

- Add final auxiliary functions and system improvement:
    1. Admin deactivate/activate user accounts
    2. Admin export recruitment data as CSV
    3. TA delete pending applications
    4. System password modification
    5. AI-based job recommendation for TAs
    6. Full system logging, bug fixes & UI/UX refinement

Deliverables:

- Working Software version 4 (complete system)
- Demonstration video & final report
- Final viva & full GitHub repository

4. Prioritisation Methods

User stories and features are prioritised using a combination of MoSCoW Method and Value-Effort Matrix, with explicit priority labels (Highest/High/Medium/Low) assigned in the Product Backlog. Prioritisation is guided by user value (core vs. supplementary), role criticality (TA/MO vs. Admin), and business necessity (must-have vs. nice-to-have).

4.1 Primary Method: MoSCoW Classification

Stories are categorised into four MoSCoW tiers, directly mapped to the Backlog's priority labels:

- **Must-Have (Priority = Highest):**

Core MVP features critical to the TA recruitment lifecycle; without these, the system is non-functional.

Includes: System Login, System Logout, Create TA Profile, Post TA Job, Browse Open Jobs, Submit Application, View Applicants, TA Submit Resumes Online.

- **Should-Have (Priority = High):**

Enhanced features that improve recruitment efficiency and system management.

Includes: Review Applications, Check Application Status, View All Jobs (Admin), Edit/Close Job, View TA Workload (Admin), MO Recruitment info publishment.

- **Could-Have (Priority = Medium):**

Convenience features that improve UX and support daily operations.

Includes: TA Registers and Logs in, TA Saves Favorite Positions, Favorite Jobs, Post Job Announcement, Manage User Accounts (Admin), Filter Applicants by Skill (MO), AI Balance Workload (Admin).

- **Nice-To-Have (Priority = Low):**

Nice-to-have features non-critical for initial launch, implemented last.

Includes: Deactivate User Accounts (Admin), Export Recruitment Data (Admin), Delete Pending Application (TA), Change Password, AI Job Recommendation (TA).

4.2 Secondary Method: Value-Effort Matrix

For stories in the same MoSCoW tier, the Value-Effort Matrix is used to rank delivery order:

- High Value, Low Effort: Delivered first (e.g., Browse Open Jobs, Logout, Change

Password).
- High Value, High Effort: Scheduled early with dedicated resources (e.g., Submit Application, View Applicants, Review Applications).
- Low Value, Low Effort: Delivered in later Sprints (e.g., Delete Pending Application, Export Data).
- Low Value, High Effort: Delivered last (e.g., AI Job Recommendation, AI Balance Workload).

4.3 Additional Prioritisation Criteria

- Role-Based Value: Features for TA and MO are prioritised over Admin-only functions.
- Dependency Order: Core functions (login, profile, job posting) are delivered before dependent functions (application, review, statistics).
- Non-Functional Requirement (NFR) Alignment: Security-critical features (login lock, session control) are prioritised early.

5. Estimation Methods

User story effort estimation is conducted using a story point estimation method with a 3-point and 5-point scale (3 = low complexity, 5 = high complexity) — the only two estimation values used in the Product Backlog. Estimation is performed by the development team via planning poker and aligned with task decomposition and complexity factors.

5.1 Primary Method: Story Points (Relative Sizing)

- **Scale Definition**: A fixed 2-level scale is used for simplicity (no arbitrary linear scale), based on the **complexity of business logic, integration, and data handling**:
    - **3 Story Points**: *Low Complexity* — Single-role functionality, simple business logic, no cross-object integration, direct data read/write from TXT/CSV files (e.g., *US-SYS-001 System Login*, *US-CORE-001 Create TA Profile*, *US-MID-001 Change Password*).
    - **5 Story Points**: *High Complexity* — Cross-role/cross-object integration, complex business rules, data aggregation/calculation, or AI/automation logic (e.g., *US-CORE-004 Submit Application* [links TA/Job/Application], *US-MID-003 AI Job Recommendation* [skill match algorithm], *US-CORE-008 View All Jobs (Admin)* [data aggregation]).

5.2 Estimation Drivers (Complexity Factors)

Each user story's estimation is based on three key complexity drivers, all tied to the project's technical and business constraints:

- **Object Integration**: Number of core system objects (TA/Job/Application/ User) the story interacts with (more integration = higher complexity).
- **Business Logic Complexity**: Number of rules/constraints embedded (e.g., duplicate application blocking, account lock after 3 failed logins, skill match calculation).
- **Data Handling**: Type of data operation (simple read/write vs. aggregation/calculation/AI processing) and adherence to TXT/CSV file format constraints (no database).

5.3 Estimation Validation

- **Task Decomposition**: High-complexity stories (5 points) are decomposed into smaller actionable tasks (e.g., *US-CORE-004 Submit Application* is split into: generate unique Application ID → link TA/Job ID → block duplicate applications → save to CSV).
- **Historical Reference**: Estimations are validated against similar completed stories (e.g., *US-CORE-005 View Applicants* [5 points] uses the same cross-object integration logic as *US-CORE-004 Submit Application*).
- **No Time-Based Estimation**: Story points represent **relative effort**, not calendar time — the fixed Sprint timeline (30/3/26) is used to allocate stories to Sprints based on total story points per Sprint (balanced 3/5 point distribution).

2.Supporting material

1.Interview Record

During the final class on March 13, members of Group 28 conducted interviews and discussions with the course instructor and several TAs. We have summarized the key questions and information as follows:

1.1 We asked the TAs about the current way they apply for the TA position.

The TAs described and showed that the current recruitment method is through questionnaires, which is very cumbersome for both TAs and course administrators and has led to several problems.

1.2 We also asked the TAs which functions of the software are essential and can solve the current inconvenience of the questionnaire system.

(1) The TAs mentioned that they currently do not know how many TAs are needed for each course, which often leads to too many applicants for one course. In addition, they do not have access to detailed course information. We suggest that the software should clearly display the number of TAs required and available positions for each course, and the MO should publish detailed course information.

(2) The TAs also pointed out that they cannot know whether their applications have been approved. If they are only notified of the final result at the end, it lacks flexibility. We discussed with the TAs and agreed that the application status should be clearly displayed. There will be an application deadline. After the deadline, if the MO approves or rejects an application, the TA will be notified through the software, so that TAs can know the result early and arrange their time reasonably.

(3) We also discussed with the TAs whether applicants can set priorities for the courses they want to work for, similar to the college application system in mainland China, such as choosing whether to accept adjustment. This may be helpful for both the MO's selection process and the TAs' work motivation.

1.3 We discussed the resume upload issue with the TAs and had some different opinions.

(1) The TAs said that PDF resumes are the most mainstream and commonly used format and also the requirement of this project. They hope the system can support PDF resume uploads. Moreover, many existing PDF resumes can be used directly, which is very convenient for TAs.

(2) Some students from our group and another group suggested building a standardized resume system inside the software. Information can be collected once and reused or updated later. However, the TAs said this function is not very practical, so we will keep this as an alternative plan for now.

(3) The TAs gave us some suggestions on PDF resumes. Nowadays, many models are multimodal and have excellent performance in intelligently recognizing resume content. Therefore, they recommended that we introduce a model to assist the MO in reviewing resumes, which would be a plus for the project.

1.4 We further solicited core demands for the recruitment process from the course instructor.

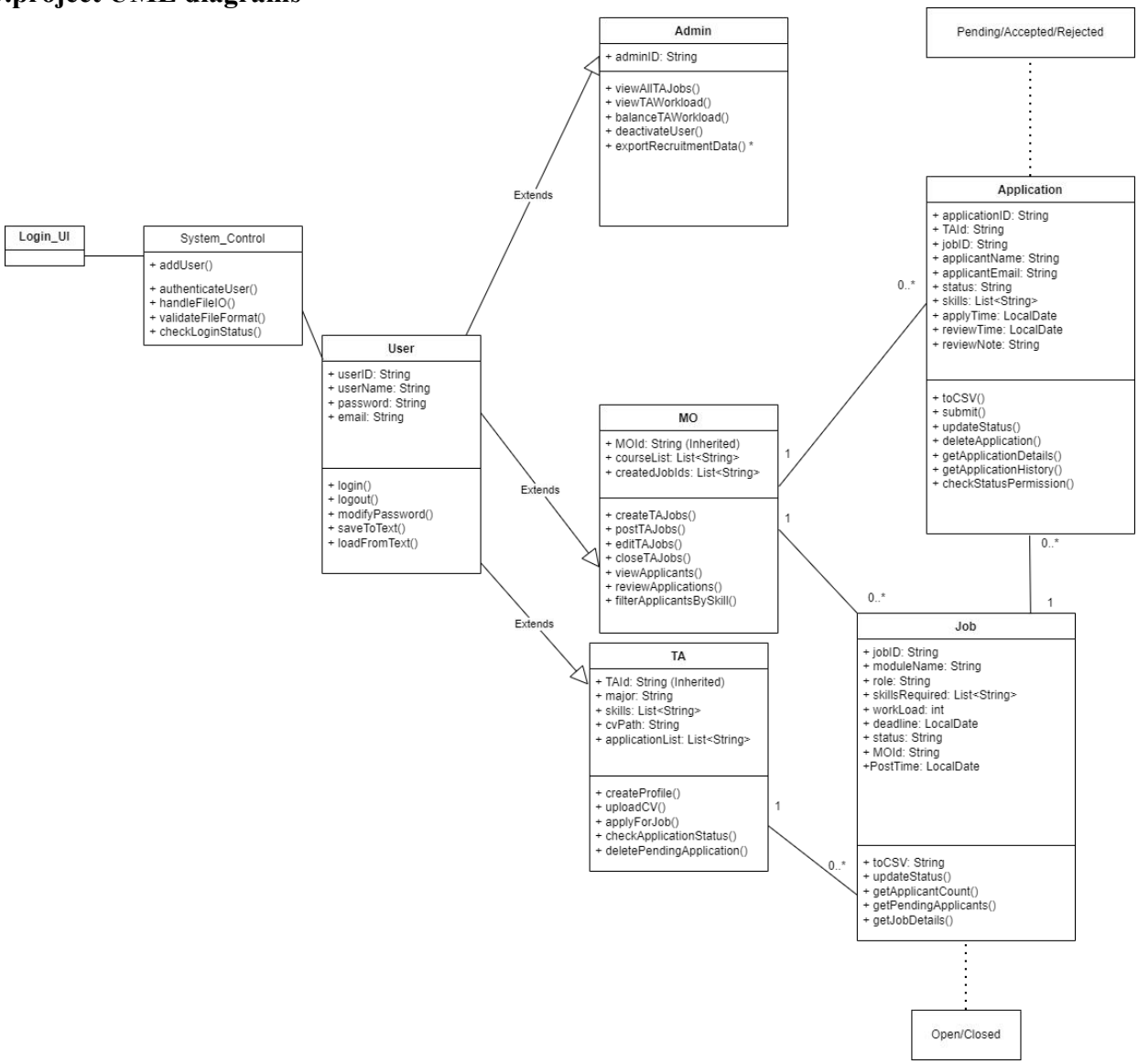
(1) Different TA positions correspond to different course characteristics, so MOs need to add specific professional background or skill requirements for each TA recruitment position in the system, and the system will display the requirements in a prominent position on the job posting page to help TA applicants achieve accurate position matching. However, the recruitment diagram is standard for every position.

(2) If the number of recruited TAs for a course fails to meet the set quota after the first application deadline, the system supports MOs to reinitiate the recruitment process and reopen the application channel for the position; the newly set application time and recruitment reminder will be synchronously displayed on the job browsing page to inform all TA applicants.

(3) The instructor put forward a clear request for an automatic notification function, hoping that

the system can realize real-time automatic push of important recruitment information, including application status approval/rejection results, reopening of recruitment channels, etc. The notification will be displayed in the personal center of the relevant users (TA/MO) to replace manual notification and improve the efficiency of the recruitment process.
(4) The system will add a highlighted display module for the application deadline of each TA position on the job posting detail page to prevent applicants from missing the application time; at the same time, MOs are required to fill in and publish the specific TA working hour when posting jobs, so that TA applicants can make rational application decisions combined with their own time schedule.

2. Benchmarking Reference Websites

https://www.opencats.org/
https://www.jobs.ac.uk/categories/university-teaching
https://www.mgmt.ucl.ac.uk/vacancies/teaching-assisstant

3.project UML diagrams